

Blocked Matrix Multiplication

Omar Attaalla

May 2025

1 Introduction

Matrix multiplication has always been an incredibly important function in the world of science and engineering. In particular, in recent times, the speed at which matrix math is performed has been incredibly important. With the rise of artificial intelligence and machine learning, the benefit of improved speed could lead to faster discoveries, reduced costs, and more productivity. This paper discusses one solution to faster matrix multiplication by improving parallelization and memory constraints.

Blocked matrix multiplication breaks matrices into smaller blocks that are then multiplied together to produce a resulting matrix. This method for multiplying matrices leads to a very important consequence. Since CPU's are often memory constrained with a hierarchical cache layout that is very limited in size, smaller blocks reduce the time spent on reading and writing to memory. Additionally, when using multiple cores (and assigning blocks to each core) we also find the added benefit of easier cache coherence. Since we are using smaller blocks, we achieve better locality. In this paper, we show an implementation of blocked matrices that work with any matrix and block sizes.

2 Implementation

This implementation is modified from the block matrix algorithm written during lecture. To accommodate any block and matrix size, we use an alternate partitioning sizes that computes the multiplications on the remaining cells (even if they are not a multiple of the block size). This leads to the following multiply method:

```
void multiply_blocked(double ** A, double ** B, double ** C, int m_low,
                    int m_high, int m, int n, int p, int blocksize) {
    for (int kk = 0; kk < n; kk += blocksize) {
        for (int jj = 0; jj < p; jj += blocksize) {
            for (int i = m_low * blocksize; i < std::min(m_high * blocksize, m); i++) {
                for (int j = jj; j < std::min(jj + blocksize, p); j++) {
                    double sum = C[i][j];
                    for (int k = kk; k < std::min(kk + blocksize, n); k++) {
                        sum += A[i][k] * B[k][j];
                    }
                    C[i][j] = sum;
                }
            }
        }
    }
}
```

We utilize `std::min` to ensure that we do not go out of bounds and read all possible cells for the multiplication. However, this means that choosing the block size is very important since an incorrect block size will reduce the amount of parallelization that occurs. The exact block partitions are then adjusted as well, where each thread is assigned a number of blocks:

```
int numblocks = (m + blocksize - 1) / blocksize;
int threadblocks = (numblocks + numthreads - 1) / numthreads;
int lowindex = i * blocks_per_thread;
int highindex = std::min((i + 1) * blocks_per_thread, numblocks);

threads.push_back(std::thread(multiply_blocked, A, B, C,
                             lowindex, highindex, m, n, p, blocksize));
```

The entire implementation and the test-bench is included with this report.

3 Experimentation and Analysis

3.1 Setup

In order to test the performance differences between a blocked and non-blocked implementation of matrix multiplication, we constructed a test-bench that includes varying block sizes, thread counts, and matrix sizes. This ensures we get a full picture when one solution is better than the other. For our testing, we used square matrices with varying sizes from 100x100 to 1600x1600. We utilized a machine with an AMD x86 Ryzen 7 5800H that contains 16 cores. Thus, we altered the number of threads from 1 to 16. We also altered the block sizes from 8x8 to 512x512 to gauge performance changes when the block size is changed.

3.2 Results and Analysis

These experiments led to some interesting results and we can see a clear speed-up when using blocked matrix multiplication in certain scenarios. Table 1 below shows the results of fixing the block size to 128x128, varying the size of the matrices from 100x100 to 1600x1600.

Table 1: Blocked vs Non-Blocked Computation Times (s)
 blocked : non-blocked
 128x128 block size

m	Threads=1	Threads=2	Threads=4	Threads=8	Threads=16
100	0.0010221 : 0.0014617	0.0011548 : 0.0006686	0.0011078 : 0.000509	0.0011482 : 0.0004923	0.0011998 : 0.0009507
200	0.0073681 : 0.0076013	0.0047936 : 0.0036759	0.0046672 : 0.0022466	0.0049921 : 0.0015222	0.0047114 : 0.0016496
400	0.0606965 : 0.0585533	0.0459885 : 0.0289014	0.0196989 : 0.0219897	0.0199256 : 0.0123468	0.0197426 : 0.0089446
800	0.451836 : 0.471584	0.293219 : 0.25244	0.152293 : 0.127707	0.0909557 : 0.0953999	0.0925179 : 0.0699279
1600	3.6861 : 11.7185	2.09139 : 6.12683	1.22193 : 3.19303	0.735337 : 1.74635	0.479611 : 1.44111

We can see that blocked matrices perform much better when the matrix size is large. When $m=1600$ with 16 threads, we found that using blocked matrices led to over a 300% speed-up. In general, when the matrix was large, blocked matrices were much faster since we are not constrained by the limited cache size and memory bandwidth. However, for small matrices, we found that the overhead from using blocked matrices led to a slow-down.

The block size did drastically impact performance as well. Table 2 below shows the same test where we

fixed the block size to be 512x512.

Table 2: Blocked vs Non-Blocked Computation Times (s)
 blocked : non-blocked
 512x512 block size

m	Threads=1	Threads=2	Threads=4	Threads=8	Threads=16
100	0.0010349 : 0.0013261	0.0012299 : 0.0006695	0.0011095 : 0.0004691	0.001039 : 0.0005246	0.0011585 : 0.0009286
200	0.0082354 : 0.0088459	0.0081866 : 0.0039523	0.0078381 : 0.0023199	0.0073741 : 0.0016065	0.0073942 : 0.0018715
400	0.05799 : 0.0583095	0.0589625 : 0.0291984	0.0584019 : 0.0147282	0.0573659 : 0.0091309	0.0591815 : 0.0078256
800	0.472665 : 0.459973	0.301851 : 0.249278	0.325351 : 0.12935	0.30397 : 0.0908718	0.302328 : 0.075807
1600	3.79743 : 11.5518	2.4537 : 6.207	1.2776 : 3.04824	1.28633 : 1.77375	1.32067 : 1.40076

We can see that when the block size is larger than or close to the size of the matrix, the performance drops significantly. Even for $m=800$, when the block size is fixed to 512x512, we saw that using blocked matrices was 428% slower than performing the normal matrix multiplication. This follows since if we are using large blocks, we do not gain the benefit of the multiple cores (one core does most of the work). This is also true because we do not benefit from the smaller memory usage of using blocked matrices. Thus, we are also hurt by the limited cache size and memory bandwidth.

The full test-bench results are included with this report. We saw the same trend occurring, where the block size was important in the performance difference between block matrix multiplication and normal matrix multiplication. We can also see how drastic parallelization improves the performance (even without using blocked matrices).

4 Conclusion

In this paper, we discussed blocked matrix multiplication and introduced one such implementation that supports any block/matrix sizes. We did this by bounding the ranges of blocks and performing the multiplication on remaining cells if there is an uneven block. In general, we found a significant speed up when using blocked matrices, especially for larger matrices (over 3x speed-up). However, the size of the block was very important, and we found that the optimal block size was less than that of the matrix size and is a factor of the number of threads we have.